A 3D Puzzle Adventure Game

Graphics Unity Project 2024

Deadline: 27/06/2024

Puzzle Adventure: Mystery Tale 3D game for Android.

Legend of the Skyfish 3D game for PC.

Contents

1 Introduction

The task is to create a single-player-like Game in **Unity** game engine.

A fully completed game (100% of marks) should consist of the following sections and Unity components:

1. **Graphics**: 3D scene design, 3D objects, textures, materials, lights.
2. **Physics**: colliders, rigidbodies, forces.
3. **Scripting**: game logic and mechanics, event handling.
4. **Audio**: sound effects, music.
5. **UI**: text display, buttons using Unity's UI components.

2 Game Overview & Core Components

The concept of the Graphics Course's project for 2024 is to develop a game that is associated adventure and puzzle genre and it's up to you to implement your game in whichever way you see fit. However, **the game must abide by certain rules** which will be explained in detail in the sections below.

What is an adventure puzzle game?

An adventure puzzle game is defined as an adventure game, filled with puzzles and logic riddles. In this genre, players navigate through immersive environments. The players are tasked with moving objects or destroying obstacles to get to the next area. In certain games, the players are tasked with finishing the task before a timer has expired. In other games, the players have unlimited time to finish their puzzles but they also have to face enemies with AI called non-playable characters (NPCs) which attack and drain their healthbar. In certain games, the game logic can spawn boosters in the ground which the player can get to their inventory and replenish their lost healthbar. This also applies to games with a time interval where the boosters increase the time the player is allowed to solve the puzzle.

2.1 Playable and Non-Playable Characters

The game will contain the following characters:

- The **Playable Character (PC)**. The PC is a fictional character, whose actions are controlled by the player. The PC needs to pass through several levels (scenes) to fulfill the goal of the game. The PC enters one scene and needs to complete several tasks and move to a dedicated exit to progress to the next scene. In his/her way there might be physical obstacles, while the PC may interact with objects and non-playable characters (NPCs). The PC has a healthbar and a basic inventory. The PC can get boosters from the scene, which can be added to his/her inventory.
- **Non-Playable Characters (NPCs)**. Those are game objects with some basic form of artificial intelligence (AI). NPCs can be of different types and have various interactions. E.g. they can interact with the PC, by helping or attacking him/her. However, all NPCs must have boundaries based on the type of the aspects of your game.

2.2 Interactable and Non-Interactable Objects

The game will contain the following type of objects:

- **Interactable objects**. Like any puzzle game, there must be some objects that the player can interact with. Those can be movable rocks, unlockable doors, opening chests, growing plants, etc. The task of the player is to interact with those objects.
- **Physical obstacles**. Those are immovable objects that block the path of the player (e.g. trees, rocks, etc.).
- **Boosters**. Boosters are located somewhere in the scene. The playable character can grab and use them, or put them in his/her inventory for later use e.g. restore part of their healthbar or increase the remaining time.

2.3 Commands

The player must be able to use specific commands. In total, 4 commands must be implemented:

- **Move Command**: The player can move the playable character to a specific destination. This movement can be done either with a cursor (move where I click) and/or with WASD keys.
- **Interact with Object Command**: The playable character can interact with other objects which are part of the solvable puzzle. Such examples are: grab a rock and move it, use a key on a door to open it etc.
- **Use Booster Command**: The playable character can use an item from their inventory to boost their stats. Examples are: Use potions to restore health points, use food item to restore hunger, use clock to increase remaining time.
- **Interact with NPC Command**: The playable character can interact with NPCs. NPCs can be either hostile or friendly. A classic example is the attack command where a player can attack NPCs.

Adjust these commands to your game needs and gameplay.

2.4 Level Design

The level design is a scene with a defined area, a playable character entrance and exit. The scene is filled with physical obstacles, interactable objects, boosters and NPCs. The playable character will enter from the starting position and finish on the exit. The main objective is to unlock the exit by achieving a set of conditions e.g. solving the puzzle and finding a key or a gem that unlocks the exit.

2.5 Camera Control

The player must be able to control the camera view. In particular, the player must be able to zoom in and out. Depending on the game type of your choosing it might be a good idea to have the camera follow the playable character or rotate it around an axis for better viewing angles.

2.6 AI Components

AI Navigation: The movement of your player and NPC units must have some path-finding logic that will allow them to traverse the level. An NPC can be either hostile and move towards the player, or friendly and move in a predefined path waiting to interact with the player. Both the player and the NPC cannot pass through obstacles. Be careful with how you define your walkable area so that no entity may fall through the level. You can achieve this functionality by using the Unity's NavMesh System.

2.7 Music and Sound Effects

Games usually have a music theme that plays in the background. Sound effects are added to improve game immersion.

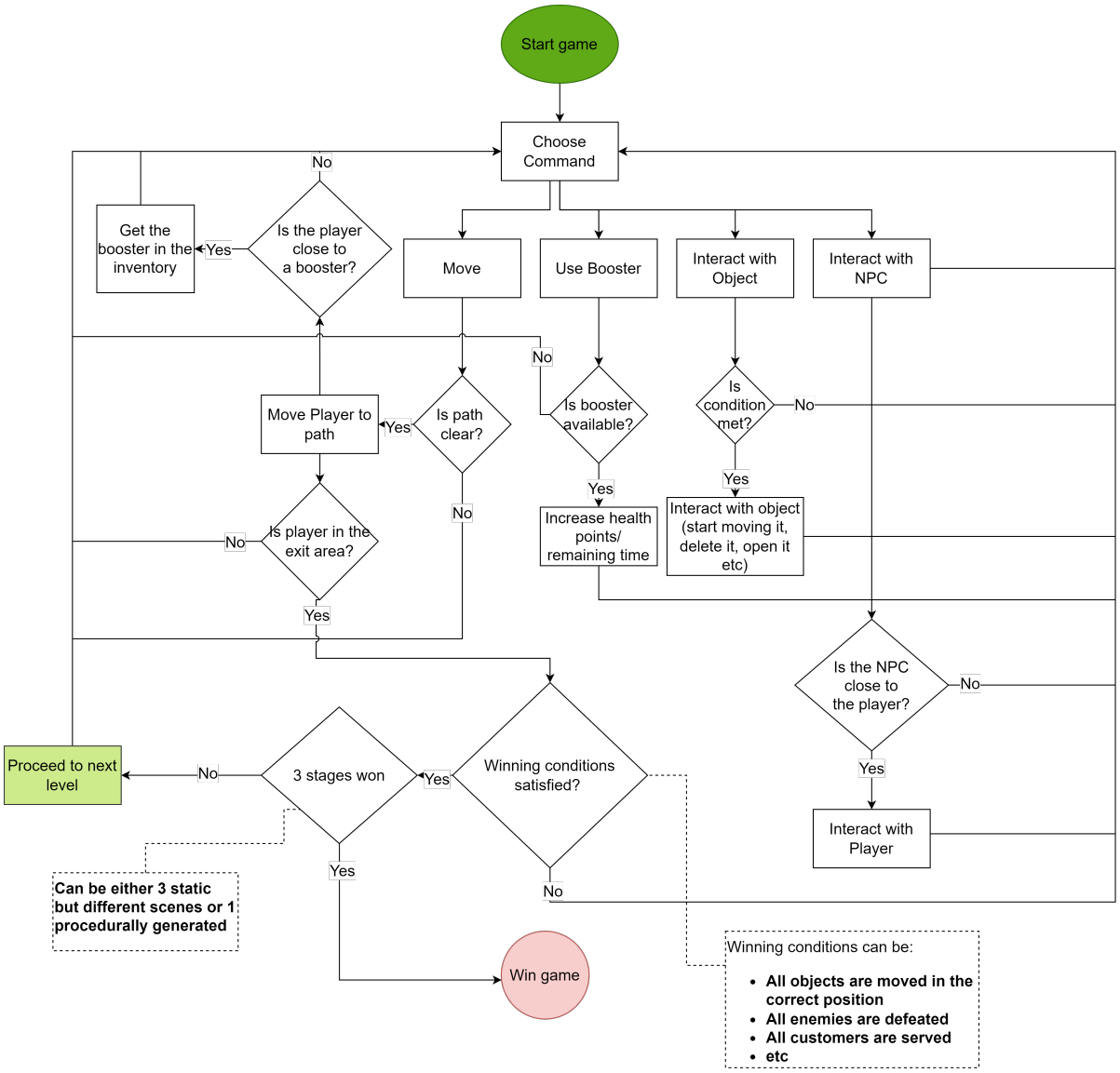
2.8 User Interface

You should implement the basic UI functionality required for your game to be played smoothly. Don't take anything for granted and explain everything in your game using UIs or by guiding your players in a smart way. Show at every stage of the game the progression to the player, and communicate with him every info that affects his/her experience throughout the entire procedure (e.g. show healthbar, available boosters, etc).

3 Game Logic

Below is a flowchart that can work as a template for your game logic design.

You may open a unique copy of this template in your browser and edit it (functionality provided by draw.io) by following this link: here



Game Logic Template

4 Game Design Document

The core game design dictates what the player expects to see in the game in three sentences. It is a very broad guideline that dictates the core design principles of the development process and helps guide us when coming up with new ideas or when searching for assets.

The core design includes three topics: Playable Character (Who is the character controlled by the player?), Setting (In what location and time period does the game take place? What is the main visual theme or technological background?) and Goal (what does the player seek to achieve and how?).

For puzzle adventure games, another thing included during game design is a unit/building look

up table. There, each unit's capabilities are explained in a plaintext format (see examples below).

4.1 Example game – Ancient Egypt

- **Playable Character**: The player controls a tomb rider character located in ancient Egypt.
- **Setting**: Early 90s. African desert.
- **Goal**: The player must solve puzzles involving statues in order to find the hidden treasure which is located in the final level. The player has to move all the statues in the correct pavement in order to progress to the next level. When all statues are placed in the correct pavement the exit is unlocked and the playable character can move to the next level.
- **Scenario**: The playable character is constantly chased by mummies (hostile NPC). The playable character holds a torch. The brightness of the torch runs out both over time and if the playable character is attacked by mummies. If the torch is extinguished the player loses the game and has to start over. There is an egyptian priest (friendly NPC) who heals the playable character when he is near the playable character.

Hostile NPC: Mummy	Monster that tries to attack the playable character.
Friendly NPC: Egyptian Priest	The priest heals the playable character when he is nearby.
Interactable Object: Statue	A statue that can be moved by the playable character. Must be placed in the correct pavement.
Obstacle: Palm Tree	A tree that blocks the movement of the playable character.
Obstacle: Rock	A rock that also blocks the movement of the playable character.
Obstacle: Entrance	The entering point of the playable character to the scene.
Obstacle: Exit	The exiting point of the playable character from the scene.
Booster: Torch	The torch extends the remaining time of the playable character.

4.2 Game logic

You need to include in your Game Design Document a Game Logic flowchart adapted to your own game requirements. See section 3 3.

5 Requirements

You must complete the implementation of minimum requirements first, in order to start the implementation of the extra requirements.
You are not allowed to use game-ready plugins/packages that implement the functionality of components that are listed below. For example you can not use a package that does the whole navigation system by itself for you, but you can use game-ready 3D Model assets and animations. You can use UI assets for UI/buttons but not a whole game-ready UI system package.

5.1 Basic Requirements (50%)

In order to pass the graphics course (5/10) you have to implement at least the minimum requirements.

Game design document: In order to pass the project of Graphics 2024 it is mandatory for your report to contain a game design document. **You cannot pass the course without it.**

1. **Graphics (10%)**: One (or more) Unity 3D scene(s) with the necessary 3D objects. All objects must have different materials with at least one basic texture on them. You can use Unity's 3D models (cubes, spheres, ..) to create your scene by transforming them to give them the shape you wish.
2. **Physics (5%)**: Objects must have colliders and rigidbodies (when needed) so that they behave as naturally and realistically as possible.
3. **Level Design (10%)**:
   - Three different playable levels. The difference between scenes is the location of the NPCs, the boosters and the obstacles. The puzzle logic can remain the same.
4. **Game logic and mechanics (10%)**: At the minimum requirements, your project must be able to provide the following functionality:
   - A playable character that moves in the plane.
   - At least two types of boosters that increase some useful stat of the player.
   - At least one NPC that can interact with or attack the player.
   - A game manager who will be monitoring the game logic state and check whether finishing conditions are met.
5. **UI & Cameras (10%)**: The UI components need to be handled by scripts so they are updated in real-time.
   - Main menu: When the game first opens, it should start with a main menu with at least one button to start the game and another to quit the game.
   - A **UI Credits** text object, which displays the Credits. Credits should contain all the 3D Objects', Music's and other components' sources used in your Project.
   - At least one UI text object presenting useful information about the playable character (e.g. the healthbar and the available boosters in the inventory).
   - At least one **Camera** exists in the scene.
6. **Audio (5%)**: At least one **Audio source** playing either an ambient sound.

5.2 Extra Requirements (50%) + Bonus(10%)

You must complete the implementation of minimum requirements first, to start the implementation of the extra requirements.

1. **Graphics(10%)**:
   - **Animations**: The PC and NPCs must have some basic animations for walking and interacting with the player. E.g. attacking or trading with the player, being happy with the food, executing a transaction etc.
   - In addition to the minimum requirements, you should search third-party 3d models (found on the Internet for example from Sketch-fab or Unity's Asset Store) or create your own 3d models in separate 3d-modeling software (such as 3D Studio Max, Blender, etc) and import them in your project to build your scene.
2. **Level Design (10%)**:
   - Procedural/Fully Random Generated Maps.
3. **Game logic and mechanics (20%)**:
   - More than one type of NPC with different mechanics.
   - **NPC AI**: The NPCs (agents) must have some sort of AI so that they can know how to navigate in your game area as well as be able to stop and interact with the player.
   - At least one more type of booster.
4. **UI & Cameras (5%)**:
   - More than one camera point of view.
5. **Audio (5%)**: Sound effects when something happens (e.g. opening a door, cutting carrots, hitting an enemy).

Any Extra Functionality/Effort (BONUS 10%):

- Bonus can also be given in cases where the implementation of a feature is extremely well-developed or very creative/unique.
- Self made 3D/2D assets, Music, Videos count as extra effort and give extra bonus. You must clearly state which assets were custom made and the software you made them with in your credits.

6 Basic Implementation Instructions

6.1 3D Models

You can create your own 3d models in separate 3d-modeling software (such as 3D Studio Max, Blender, etc) and import them in your Unity project. However, if you don't have previous experience with 3D modelling, this option is not recommended. For Basic shapes and level designs you can use the ProBuilder Unity package, to create simple shapes within unity editor.

You can import third-party 3d models found on the Internet (Sketchfab) or Unity's Asset Store to help you build your scene. Remember: If you download and import third-party 3D models you should include them in Credits.

You must add appropriate **materials** to object surfaces to give them a realistic feel and look. Use Unity's Material Editor accordingly.

6.2 Physics

6.2.1 Collision between objects

Objects must have **Collider** components attached to them, otherwise they will pass through each other. It is important that the collider matches the 3D shape of an object as best as possible. For example, a box collider is ok for a box-shaped object, such as a wall or a smooth floor. More complicated 3D models require different types of colliders. For example, using a box or a sphere collider for a pin is wrong, as it will result in un-realistic behaviour.

Documentation - Collision between objects

6.2.2 Rigidbody

Rigidbodies enable your GameObjects to act under the control of physics. The **Rigidbody** can receive forces and torque to make your objects move in a realistic way. Any GameObject must contain a Rigidbody to be influenced by gravity, act under added forces via scripting, or interact with other objects through the NVIDIA PhysX physics engine.

A rigidBody is defined by many parameters, such as its mass, drag, angular drag and more.

Documentation - Rigidbody

6.2.3 Physic material

The Physic Material is used to adjust **friction** and bouncing effects of colliding objects.

To create a Physic Material select *Assets, Create , Physic Material* from the menu bar. Then drag the Physic Material from the Project View onto a Collider in the scene.

Documentation - Physic Material

6.2.4 Realistic physics

Game objects affected by physics (rigidbodies) have parameters, such as mass, drag and angular drag. Set their values accordingly by experimenting, so that they behave realistically.

6.3 Scripting: Game logic and functionality

Scripts control the entire functionality of your game, such as handling user input, triggering events upon specific actions, keeping score, moving objects and more.

Your scripts will have to:

- **Initialize** variables, load game objects and/or other scripts that need to be accessed.
- **Keep track of the game state** and act accordingly. Check for key presses, position changes, collisions and more...
- **Reset game objects' state** (such as position, rotation, gravity, ...) **when required**.
- Update the UI elements when necessary. For example, have a UI text that shows the current gold the player has.

Important things to remember:

Code in Unity is written in C#. Every new script **derives** from the *MonoBehaviour* base class by default. A script deriving from *MonoBehaviour* enables the call of the Start(), Awake(), Update(), FixedUpdate(), and OnGUI() functions and has the following properties:

- A script will never be executed, unless it is attached to at least one game object in Unity. Furthermore, a single script can be attached to one or more game objects. In this case, **be careful**, as the same script will run as many times as the number of game objects it has been attached to.
- One GameObject can have multiple scripts attached, too. In this case, **be careful** of possible race conditions. (a race condition can occur when, for example, 2 different scripts try to move the same object at the same time).
- Game objects (or other scripts) need to be initialised in the Start() function of the script before use, unless the script is attached to them, in which case they can be accessed directly.

The links contain useful tutorials on the communication between scripts and Game Objects in Unity:
Link

6.4 UI and Audio

6.4.1 User Interface (UI)

Unity's UI system provides a variety of tools to render elements, such as text or images, on the screen or at a specific position in the 3D space. The UI system can be used for many different purposes.

Documentation - User Interface

It is totally up to you to choose the design, position and functionality of the UI system - as long as it meets the minimum requirements.

6.4.2 Audio

You can use Unity's audio components to play sounds in a loop(ex. background music) or if a condition is true.

Documentation - Audio

7 Useful tips

- It is highly recommended to use the Unity Documentation as it provides both explanations on all of Unity's tools as well as examples on how to use them. When you open the Documentation remember to select your Unity Version on the top left.
- It is highly recommended to give your executable version of your game to be tested by others without your guidance. Try to evaluate your game utilizing the thinking-aloud method. In a thinking-aloud test, you ask test participants to use the system while continuously thinking out loud — that is, simply verbalizing their thoughts as they move through the user interface.

  Don't trust your instincts on the quality of your UIs implementation. User experience is crucial during the testing of your game. Put effort into the creation of your UI/UX and make it as intuitive as possible.
- During the exam, you will be graded based on the playable version of your game. Anything you implement that cannot be shown while playing the game will not be graded. As such you should take into consideration:
  - Someone playing the game should be able to see everything you want to show in a maximum of 10 minutes. You will be able to give us advice while playing your game in order to achieve this.
- Excluding extreme circumstances we will not view your project through the Unity Editor so make sure your build is fully playable! Some common issues that may appear include the following:
  - UI elements may not scale correctly with your screen resolution and be out of the playable field. Make sure your game plays correctly at least for the recommended resolution of Full HD (1920x1080) or smaller. If you have a specific resolution you tested your game at and it works correctly other than Full HD specify it on your report.
  - Sometimes things that work in the Editor do not work the same way after you build the game. Always build and test your game during development! The most common issues involve the initialization of objects while switching between scenes e.g. when transitioning from the main menu to the game or when restarting the game and reloading the same scene.
- **Using a piece of code that you cannot explain will not give you any grade for that specific section!!!** It is recommended you watch online tutorials and even exchange ideas with each other but plagiarism is strictly forbidden. Additionally, custom and creative solutions that work will receive bonus grades. This means that if you get some piece of code from the internet and you cannot explain it, if you put additional effort in another field you can compensate for the grades you lost and still get a good mark.

8 Project Guidelines

- Your code must be organized in different scripts, depending on the functionality. **Each script must have a distinct role.**
- Use script names that make sense and attach them to related game objects. For example, create a "UIManager" script to handle the game UI, a "GameManager" script to hold variables and objects that keep track of the game's state.
- Your code must be written in C# programming language ONLY.
- Organize your game objects in a hierarchy that makes sense and is easy to understand. For example, if you have game objects such as "Cube(1)", "Cube(2)", "Cube(3)", ... , make them children of a single "CUBES" game object.
- Organize your asset files (textures, 3d models, materials, scripts, sounds, ...) in separate folders. Obviously, the name of the folder should be representative of the content.
- You may use the Internet (YouTube, blogs, ..) to your help for tutorials and code examples, but copy-pasting entire code or functionality is obvious and will NOT be accepted.

and remember while doing this project to **have fun !**